SIMATS ENGINEERING

ASSESSMENT TOOL 1

Annexure A: Constraint-Based Problem Solving

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE: CSA06

Submitted by:
Name: S. Jaxon Rizal
Reg No: 192511421

Code of Conduct:

*I, **S. Jaxon Rizal / 192511421**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

_____

Signature of the student

Q1. Job Scheduling (Backtracking / NP-Hard Problem)

PROBLEM DEFINITION & CONSTRAINTS

The Job Scheduling problem requires assigning a set of jobs to a group of employees. Each job has a specific duration, a strict deadline by which it must be completed, and a profit value associated with its completion. The system must operate under the following core constraints:

- **Temporal Constraints:** A job must be fully executed before its deadline expires. If a job starts too late, it violates this constraint and yields zero profit.
- **Resource Constraints:** No single employee can handle more than one job at any given moment. An employee is "locked" for the duration of the assigned job.
- **Completeness Constraints:** The objective is to maximize total profit, meaning the algorithm must seek an assignment mapping that prioritizes high-value tasks while maintaining schedule feasibility.

CONSTRAINT ANALYSIS & PRIORITIZATION

The constraints interact antagonistically: maximizing profit often requires scheduling high-value jobs, which might have conflicting deadlines or require overlapping resource allocations. To efficiently navigate this, constraints must be prioritized using a greedy heuristic prior to backtracking. By sorting the jobs in descending order of profit, the algorithm attempts to satisfy the highest-value constraints first. If two jobs have the same profit, the one with the earlier deadline is prioritized to free up resources sooner.

SOLUTION DEVELOPMENT (BACKTRACKING)

The state space is explored using a recursive function. The algorithm maintains a simulated timeline for each employee. For each job in the sorted list, the algorithm iterates through the available employees. It checks if an employee is free to start the job such that the job finishes before its deadline. If feasible, the algorithm assigns the job, updates the employee's timeline, adds the profit to the running total, and recurses to the next job. Upon returning from the recursion, it backtracks by un-assigning the job, reverting the timeline and profit, and attempting to assign the job to the next available employee or skipping it entirely.

PRUNING TECHNIQUES

To prevent exploring the entire state space, Branch-and-Bound pruning is integrated. A global tracking variable records the maximum profit found so far. At any node in the search tree, the algorithm calculates the theoretical maximum remaining profit (the sum of profits of all unassigned jobs). If the current profit plus this theoretical maximum is less than or equal to the maximum profit already found, the entire branch is pruned. This prevents the algorithm from wasting time on schedules that cannot possibly outperform the current best solution.

> **Complexity & Justification:** The worst-case time complexity is $O(E^N)$, where E is the number of employees and N is the number of jobs. By combining greedy sorting for early optimal discovery with rigorous branch-and-bound pruning, this solution aggressively trims the search space, ensuring all constraints are satisfied efficiently while guaranteeing an optimal maximum-profit schedule.

Q2. Exam Timetabling (Graph Coloring / NP-Complete Problem)

PROBLEM DEFINITION & CONSTRAINTS

The university exam timetabling problem is a classic resource allocation challenge. The goal is to schedule a set of exams into a limited number of time slots and rooms. The problem is dictated by the following strict constraints:

- **Conflict Constraints (Hard):** No student can be scheduled to sit for two different exams simultaneously. This is an absolute constraint.
- **Capacity Constraints (Hard):** The number of students scheduled in a specific room during a single time slot cannot exceed the room's physical seating capacity.
- **Temporal Constraints (Soft/Optimization):** The schedule should utilize the minimum possible number of time slots to condense the exam period.

CONSTRAINT ANALYSIS & PRIORITIZATION

This problem maps directly onto the Graph Coloring problem. Each exam represents a vertex in the graph. An edge is drawn between two vertices if there is at least one student enrolled in both exams (representing a conflict). The colors represent the distinct time slots. To prioritize these constraints, we use the Minimum Remaining Values (MRV) and Degree heuristics. Exams with the most conflicts (highest degree) are the hardest to schedule and must be assigned time slots first. This greatly reduces the chances of reaching a dead end deep in the search tree.

SOLUTION DEVELOPMENT (CONSTRAINT SATISFACTION)

The backtracking algorithm begins by ordering the vertices based on their degree (highest to lowest). It picks the first unassigned exam and attempts to assign it the earliest available time slot (color). The algorithm then checks adjacent vertices. If the chosen time slot does not violate any conflict constraints (no adjacent vertex shares this slot) and the room capacity is respected, the assignment is tentatively made. The algorithm then recurses to the next most constrained exam.

PRUNING & FORWARD CHECKING

Instead of blind backtracking, Forward Checking is applied. Whenever an exam is assigned a time slot, that time slot is temporarily removed from the domains of all adjacent (conflicting) exams. If any adjacent exam's domain becomes empty (meaning it has no available time slots left), the current assignment is immediately recognized as a failure. The algorithm prunes this branch, rolls back the assignment, restores the domains of adjacent vertices, and tries the next available time slot.

> **Complexity & Justification:** The underlying Graph Coloring problem is NP-Complete, with a worst-case time complexity of $O(k^V)$ for V exams and k time slots. The combination of Degree heuristic ordering and Forward Checking ensures that the algorithm dynamically adapts to constraints, avoiding massive subtrees of invalid schedules and proving correctness by exhaustive (but intelligent) search.

Q6. N-Queens Problem (Backtracking / Constraint Satisfaction Problem)

PROBLEM DEFINITION & CONSTRAINTS

The N-Queens problem requires placing N chess queens on an $N \times N$ chessboard such that no two queens can attack each other. The constraints are derived from the movement rules of a queen in chess:

- **Row Constraint:** Exactly one queen must exist in each horizontal row.
- **Column Constraint:** Exactly one queen must exist in each vertical column.
- **Diagonal Constraints:** No two queens can share the same major (top-left to bottom-right) or minor (top-right to bottom-left) diagonal.

CONSTRAINT ANALYSIS & PRIORITIZATION

The row constraint can be satisfied implicitly by the structure of the algorithm itself. By progressing row by row, we never even attempt to place two queens in the same row. This immediately eliminates a vast portion of the invalid search space. The column and diagonal constraints must be checked explicitly at each placement step.

SOLUTION DEVELOPMENT (BACKTRACKING)

The algorithm uses a recursive depth-first search approach. It starts at row 0. For the current row, it iterates through all columns from 0 to $N-1$. For each cell, it checks if placing a queen violates the column or diagonal constraints relative to previously placed queens. If the cell is safe, the queen is placed, and the algorithm recurses to row + 1. If the algorithm successfully reaches row N, a valid configuration is recorded. It then backtracks to find all other valid configurations.

PRUNING TECHNIQUES (BITMASKING OPTIMIZATION)

To optimize the constraint validation, we use boolean arrays or bitwise operators instead of scanning the board. Three 1D arrays are maintained: `col_safe`, `major_diag_safe`, and `minor_diag_safe`. When a queen is placed at `(row, col)`, it immediately marks `col`, `row - col + N - 1` (major diagonal), and `row + col` (minor diagonal) as unsafe. This allows for $O(1)$ constraint validation. If a cell falls on any unsafe line, the iteration skips it instantly, effectively pruning that branch.

> **Complexity & Justification:** The raw search space is $O(N^N)$, but with implicit row placement and backtracking, the complexity is bounded by $O(N!)$. The $O(1)$ pruning arrays ensure that invalid placements are rejected immediately. This deterministic approach mathematically guarantees that all valid, non-attacking configurations are generated without missing any edge cases.

Q7. Sudoku Solver (Backtracking / NP-Complete Problem)

PROBLEM DEFINITION & CONSTRAINTS

Sudoku is a logic-based combinatorial number-placement puzzle. The objective is to fill a 9×9 grid with digits so that specific constraints are met. The initial grid comes partially filled. The core constraints are absolute and symmetric:

- **Row Constraint:** Each row must contain the digits 1-9 exactly once.
- **Column Constraint:** Each column must contain the digits 1-9 exactly once.
- **Subgrid Constraint:** Each of the nine 3×3 subgrids must contain the digits 1-9 exactly once.

CONSTRAINT ANALYSIS & PRIORITIZATION

The constraints in Sudoku are highly interconnected. Placing a single number restricts the available numbers in its row, column, and subgrid simultaneously. To solve the puzzle efficiently, we prioritize the cells that are the most constrained. By applying the Minimum Remaining Values (MRV) heuristic, the algorithm always targets the empty cell with the fewest valid legal numbers left to choose from.

SOLUTION DEVELOPMENT (BACKTRACKING)

The algorithm scans the grid to find an empty cell (preferably using MRV). For this cell, it iterates through the digits 1 to 9. It evaluates if placing a digit violates any of the three main constraints by checking the current state of the row, column, and subgrid. If the digit is valid, it is placed in the grid, and the algorithm recurses to solve the rest of the

board. If a subsequent placement leads to a state where no valid numbers can be placed in an empty cell, the algorithm backtracks, clears the cell, and tries the next digit.

PRUNING & CONSTRAINT PROPAGATION

Advanced solvers use Constraint Propagation (like the AC-3 algorithm). When a number is tentatively placed, it is mathematically removed from the possible domains of all peers (cells in the same row, col, or block). If any peer's domain drops to a size of 1, that number is forced, creating a cascade effect. If any peer's domain drops to 0, a contradiction is detected, and the branch is pruned immediately without further recursion.

> **Complexity & Justification:** While generalized $N \times N$ Sudoku is NP-Complete, the backtracking algorithm with MRV and constraint propagation handles standard 9×9 puzzles with near-instantaneous performance. The rigorous constraint checking ensures absolute correctness and prevents the algorithm from wandering into deeply flawed board states.

Q9. Subset Sum Problem (Backtracking / NP-Complete Problem)

PROBLEM DEFINITION & CONSTRAINTS

The Subset Sum problem asks whether there exists a subset of a given set of positive integers whose elements sum up exactly to a specified target value. The constraints governing this decision problem are:

- **Sum Constraint:** The aggregate of the selected integers must perfectly match the target sum T.
- **Inclusion/Exclusion Constraint:** Each element from the original set can be selected at most once.
- **Domain Constraint:** All elements and the target sum are strictly positive integers.

CONSTRAINT ANALYSIS & PRIORITIZATION

Because all integers are positive, the running sum is monotonically increasing as more elements are added to the subset. This property is crucial. By sorting the input set in ascending order before executing the search, we prioritize smaller increments, which provides a structured and predictable growth rate for the running sum, allowing for highly effective bounding.

SOLUTION DEVELOPMENT (BACKTRACKING)

The state space is visualized as a binary tree. At each level (representing an element in the set), the algorithm makes a binary choice: either include the element in the subset or exclude it. The recursive function takes the current index and the current running sum as parameters. If the running sum exactly equals the target, the algorithm returns true (or records the subset). If not, it branches to include the current element (adding its value to the sum) and then branches to exclude it.

PRUNING TECHNIQUES

Two primary pruning rules are applied to drastically reduce computations. **Upper Bound Pruning:** Because the array is sorted, if the current running sum plus the next element exceeds the target, we can instantly prune this branch and all subsequent branches in this path, as adding any further positive integers will only push the sum further away from the target. **Lower Bound Pruning:** The algorithm pre-calculates the total sum of all remaining elements. If the current running sum plus all remaining elements is strictly less than the target, the branch is pruned, as the target is mathematically unreachable.

> **Complexity & Justification:** The worst-case time complexity is $O(2^N)$, representative of NP-Complete subset generation. However, the pre-sorting step $O(N \log N)$ combined with aggressive upper and lower bound pruning ensures that the algorithm operates efficiently on practical datasets, bypassing vast swaths of invalid subset combinations.

Q10. Hamiltonian Path Problem (Backtracking / NP-Complete Problem)

PROBLEM DEFINITION & CONSTRAINTS

Given a graph $G = (V, E)$ representing a network of nodes (e.g., cities) and edges (e.g., roads), the objective is to determine if there exists a simple path that visits every single vertex exactly once. The constraints are deeply structural:

- **Path Continuity Constraint:** Movement from one vertex to another is only permitted if a valid edge exists between them in the graph.
- **Visitation Constraint:** Every vertex must be visited exactly once. No vertex can be skipped, and no vertex can be revisited.

CONSTRAINT ANALYSIS & PRIORITIZATION

The most restrictive constraint in finding a Hamiltonian path is the connectivity of the graph. If the graph contains isolated vertices, or if a vertex has a degree of 1 (and it's not chosen as the start or end point), a Hamiltonian path is fundamentally impossible. Prioritizing exploration from vertices with the lowest degrees (Warnsdorff's heuristic variant) forces the algorithm to secure the most restricted nodes early in the path.

SOLUTION DEVELOPMENT (BACKTRACKING)

The algorithm initializes an empty path and a boolean `visited` array. It selects a starting vertex (or iterates through all possible starting vertices). From the current vertex, the algorithm iterates through all its unvisited neighbors. It appends a chosen neighbor to the path, marks it as visited, and recursively attempts to continue the path. If the path reaches a length equal to V, a valid Hamiltonian path is found. If the path reaches a dead end before visiting all vertices, it backtracks by removing the vertex from the path and unmarking it.

PRUNING TECHNIQUES

The primary pruning mechanism is the strict enforcement of the visited array, preventing cycles. Advanced topological pruning involves connectivity checks. At any point during the backtracking, if removing the currently visited vertices leaves the remaining unvisited graph disconnected into two or more separate components, the branch is immediately pruned. It is geometrically impossible to visit disconnected components sequentially without revisiting a node.

> **Complexity & Justification:** The time complexity scales factorially, bounded at $O(V!)$ in the worst-case scenario for dense graphs. The robust constraint checking via visited matrices and connectivity heuristics ensures that valid paths are identified strictly within the defined graph topology, satisfying the NP-Complete verification criteria.

Q11. Traveling Salesman Problem (Backtracking / NP-Hard Problem)

PROBLEM DEFINITION & CONSTRAINTS

The Traveling Salesman Problem (TSP) is an optimization extension of the Hamiltonian Cycle problem. A salesman must visit a set of cities exactly once and return to the starting city. The goal is to minimize the total travel cost. The constraints are:

- **Cycle Continuity:** The path must form a closed loop, starting and ending at the same designated city.
- **Uniqueness Constraint:** Every intermediate city must be visited exactly once.
- **Optimization Constraint:** The sum of the edge weights (distances or costs) of the chosen route must be the absolute minimum among all possible valid routes.

CONSTRAINT ANALYSIS & PRIORITIZATION

The addition of the optimization constraint makes the search space significantly more difficult to navigate, as finding *a* solution is not enough; we must find the *best* solution. Constraints prioritize greedy exploration first: when branching from a city, the algorithm sorts the neighbors based on edge weight, traversing the shortest edges first in hopes of finding a low-cost baseline quickly.

SOLUTION DEVELOPMENT (BACKTRACKING)

The recursive function tracks the current city, the list of visited cities, the number of cities visited, and the accumulated travel cost. Starting from city 0, it recursively visits unvisited neighbors, adding the edge weight to the accumulated cost. When all cities have been visited, it checks if an edge exists back to city 0. If so, it adds that final edge

weight to calculate the total tour cost. It records this cost if it is lower than the best cost found so far, then backtracks to explore other permutations.

PRUNING (BRANCH AND BOUND)

Without pruning, TSP explores $O(V!)$ permutations. Branch and Bound is vital here. A global variable `min_cost` is initialized to infinity. Whenever a valid tour is completed, `min_cost` is updated. During the search, if the `accumulated_cost` of a partial path strictly exceeds `min_cost`, the algorithm halts exploration of that branch. An advanced lower bound can also be calculated by taking the accumulated cost and adding the minimum outgoing edges for all unvisited cities. If this heuristic sum exceeds `min_cost`, it prunes the branch.

> **Complexity & Justification:** TSP is NP-Hard. The brute-force time complexity is $O(V!)$. However, the implementation of Branch and Bound pruning drastically cuts down the search tree by establishing strict cost ceilings early in the execution. This ensures absolute optimality while managing the combinatorial explosion of the search space.

Q14. Vertex Cover Problem (NP-Complete / Constraint Satisfaction Problem)

PROBLEM DEFINITION & CONSTRAINTS

Given an undirected graph, a Vertex Cover is a subset of vertices such that every single edge in the graph is incident to at least one vertex in the subset. The goal is to find a vertex cover of minimum size. Constraints include:

- **Edge Coverage (Hard Constraint):** For every edge (u, v) in the graph, the vertex cover set must contain u, v, or both.
- **Size Optimization:** The cardinality (number of elements) of the vertex cover subset must be minimized.

CONSTRAINT ANALYSIS & PRIORITIZATION

The edge coverage constraint provides a rigid binary choice that dictates the structure of the search algorithm. Since every edge must be covered, picking any uncovered edge immediately forces a decision: we MUST include one of its endpoints. Prioritization involves targeting vertices with the highest degrees, as including a high-degree vertex covers a large number of edges simultaneously, driving the required cover size down faster.

SOLUTION DEVELOPMENT (BACKTRACKING)

The algorithm maintains the current graph state (edges remaining to be covered) and the current subset of vertices. It identifies an arbitrary uncovered edge (u, v). It then branches recursively into two possibilities: 1. Add vertex u to the cover subset, and remove all edges incident to u from the graph. 2. Add vertex v to the cover subset, and remove all edges incident to v from the graph. This process continues until no edges remain. The smallest valid subset found across all branches is the optimal vertex cover.

PRUNING & BOUNDED SEARCH TREES

To optimize, we apply a Bounded Search Tree approach. We maintain a variable `best_size` representing the size of the smallest cover found so far. If the size of the current recursive subset equals or exceeds `best_size`, the branch is pruned. Furthermore, if the graph contains a vertex of degree 1 (a pendant vertex), we can deterministically prune the search tree: we *always* choose to include its neighbor in the cover, as the neighbor covers the pendant edge plus potentially other edges, whereas the pendant vertex covers only the pendant edge.

> **Complexity & Justification:** The raw backtracking complexity is exponential. With bounded search techniques targeting an upper bound k, the time complexity reduces to $O(2^k \times V)$. This method rigorously enforces the fundamental graph constraint while logically discarding redundant or bloated vertex combinations.

Q16. 3-SAT Problem (NP-Complete / Boolean Constraint Problem)

PROBLEM DEFINITION & CONSTRAINTS

The 3-Satisfiability (3-SAT) problem asks whether there is an assignment of true/false values to a set of boolean variables that makes a given Boolean formula evaluate to True. The formula is structured in Conjunctive Normal Form (CNF), meaning it is an AND of clauses, where each clause is an OR of exactly three literals. Constraints include:

- **Clause Satisfaction:** Every single clause must evaluate to True. This means at least one literal in every clause must be assigned a True value.
- **Variable Consistency:** A boolean variable x_i cannot simultaneously be True and False across different clauses.

CONSTRAINT ANALYSIS & PRIORITIZATION

Constraints in 3-SAT interact intensely. Satisfying one clause by assigning True to a variable might inadvertently falsify a literal in a different clause. Prioritization involves looking for constraints that force specific assignments. Clauses that have been reduced to a single unassigned literal (Unit Clauses) represent absolute constraints that must be satisfied immediately to prevent formula failure.

SOLUTION DEVELOPMENT (DPLL BACKTRACKING)

The standard backtracking solution is the Davis-Putnam-Logemann-Loveland (DPLL) algorithm. It selects an unassigned variable and recursively explores two branches: one where the variable is True, and one where it is False. After an assignment, the formula is simplified (clauses containing the True literal are removed, and False literals are removed from their respective clauses). If the formula becomes empty, it is satisfiable. If any clause becomes empty (all its literals are False), the assignment is invalid, and the algorithm backtracks.

PRUNING TECHNIQUES (UNIT PROPAGATION)

DPLL utilizes powerful constraint-based pruning. **Unit Propagation:** If a clause contains only one unassigned literal, that literal is deterministically forced to be True. This bypasses branching entirely and cascades through the formula. **Pure Literal Elimination:** If a variable appears with only one polarity (e.g., always x_1, never $\neg x_1$)

throughout all remaining clauses, it is assigned the value that makes those literals True, safely removing those clauses without risking contradiction.

> **Complexity & Justification:** The worst-case complexity remains $O(2^N)$, cementing its NP-Complete status. However, unit propagation and pure literal pruning act as extreme constraint propagators, often solving massive practical instances in polynomial time by logically dismantling the search tree rather than blindly guessing assignments.

Q19. Bin Packing Problem (NP-Hard / Optimization Problem)

PROBLEM DEFINITION & CONSTRAINTS

The Bin Packing problem requires packing a given set of items of various sizes into the minimum possible number of identical bins, each with a fixed maximum capacity. The constraints governing the problem are:

- **Capacity Constraint (Hard):** The sum of the sizes of all items placed into any single bin must not exceed the specified bin capacity.
- **Completeness Constraint:** Every single item provided in the input set must be assigned to exactly one bin.
- **Optimization Constraint:** The total number of bins utilized must be minimized.

CONSTRAINT ANALYSIS & PRIORITIZATION

Item size acts as the primary restrictive constraint. Large items are exceptionally difficult to pack and restrict subsequent placements heavily. If small items are packed first, they might fragment the remaining capacity in the bins, leaving no room for the large items and forcing the algorithm to open unnecessary new bins. Thus, items must be strictly prioritized in descending order of their size (First-Fit Decreasing heuristic protocol) before the backtracking begins.

SOLUTION DEVELOPMENT (BACKTRACKING)

The backtracking algorithm sorts the items from largest to smallest. It maintains a list of currently open bins and their remaining capacities. For the current item, it iterates through all open bins. If the item fits within a bin's remaining capacity, it tentatively places it there, updates the capacity, and recurses to the next item. If the item does not fit in any open bin (or as an alternative branch to explore), the algorithm opens a new bin, places the item inside, and recurses. After exploring, it backtracks by removing the item and restoring the bin capacity.

PRUNING (LOWER BOUND MATHEMATICS)

To prune the search space, a theoretical mathematical lower bound is computed before execution: `Lower_Bound = ceil(Sum_of_All_Item_Weights / Bin_Capacity)`. During the recursive search, a variable tracks the `best_bins_count`. If the number of currently open bins in a branch equals or exceeds `best_bins_count`, the branch is pruned. Furthermore, if an initial greedy pass (using First-Fit Decreasing) yields a bin count that exactly matches the theoretical lower bound, the algorithm immediately terminates and accepts the solution, as it is mathematically impossible to find a better one.

> **Complexity & Justification:** This combinatorial problem possesses an $O(B^N)$ state space (where B is the number of bins). By merging descending-order prioritization with strict mathematical lower-bound pruning, this branch-and-bound variant effectively avoids generating wasteful, loose packing configurations, ensuring an optimal, capacity-respecting solution.